



T.C.
SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
NESNE YÖNELİMLİ ANALİZ ve TASARIM DERSİ TASARIM RAPORU

AKILLI TEDARİK VE LOJİSTİK YÖNETİM SİSTEMİ

B241210040 - Ahmet Faruk İKİZ
Dersi Aldığım Grup: 1.Öğretim A
Dersi Aldığım Hoca: Dr.Öğr.Üyesi Sümeyye Kaynak

SAKARYA
Mayıs, 2026
Nesne Yönelimli Analiz ve Tasarım Dersi

1. Proje sürecinde karşılaştığım problemler ve çözüm olarak uyguladığım tasarım desenleri:

Süreç	Karşılaşılan Problem	Seçilen Tasarım Deseni	Neden bu desen?
Sistemdeki önemli olayların loglanması.	Log nesnesinin sistem genelinde tek bir adet olması ve her yerden erişilebilmesi gerekiyordu. Dependency Injection ile yapsaydım çok fazla parametre ve kod kalabalığına yol açardı.	Singleton Pattern	Logger sınıfının kurucusunu private yaparak ve kendi içinde static örneğini barındırarak getInstance() metodunu çağıran herkese bu static örneği yoksa üretip sonra hep aynı örneği vermesiyle nesnenin tekliği garanti altına alınır.
Giriş yapan kullanıcı tipine göre görebileceği ve kullanabileceği menülerin ve seçeneklerin üretilmesi.	Bu menülerin oluşturulacağı kısımda devasa bir switch-case if-else yığını oluşması. Yeni bir menü veya rol ekleneceğinde çalışan kısma müdahale gerekmesi O/CP ihlali.	Factory Method Pattern	Menüleri üreten sınıf sadece IKullaniciView ve IKullaniciMenuFactory arayüzünü tanır. Sisteme yeni bir menü eklense dahi Baslat() metodu sabit kalır. Menü üretme görevi alt factory sınıflarına devredilir.
Stok sınırın altına her indiğinde birimlere haber verilmesi/ belli bir özelliğinin güncellenmesi.	Stok sınırın altında mı kontrolü yaptığın yerde email ve bildirim gönderme işini yapsaydım SRP'yi ihlal etmiş ve kod tekrarına düşmüş olurum.	Observer Pattern	Bütün güncelleme alması gereken observer sınıfları publisher'a abone edip tek bir noktadan güncelleme kolaylığı. İleride başka bir birimin daha güncellenmesi gerektiğinde sadece o sınıf da IObserver'ı gerçekler ve abone olur.
Karmaşık ürünlerin içindeki basit ürünlerin yönetilmesi ve karmaşık ürünün çeşitli field'larının hesaplanması.	Kodda tip kontrolü yapıp karmaşık ürünse alt ürünleri de hesapla şeklinde kod yazmak Liskov prensibini ihlal eder ve polimorfizm'e aykırıdır. Kodu karmaşıktırır ve yönetimini zorlaştırır.	Composite Pattern	İki ürün de IUrun arayüzünü gerçekler ve karmaşık ürün kendi içinde yine IUrun tutar. Bu sayede hesaplama yapan istemci döngü halinde IUrunlerin fiyatHesapla() metodunu çağırır. Karmaşık ürün ise fiyatHesapla() metodu

Süreç	Karşılaşılan Problem	Seçilen Tasarım Deseni	Neden bu desen?
Siparis'in yaşam döngüsü boyunca çeşitli durum geçişleri yaşaması ve davranışlarının bu durumlar karşısında değişmesi.	Durum makinesi diagramlarının gerçekleştirilmesinde switch-case veya if-else kullanımı bir süre sonra yönetilmesi zor karışık spaghetti kodlara sebep olur.	State Pattern	Siparis'in sahip olduğu her bir durum için IDurum arayüzünü gerçekleyen ayrı sınıflar açılıp siparis ve IDurum arasında aggregation (has-a) ilişkisi kurulur. Bu sayede durum üzerinden kullanılacak metotlar her aşamada farklılaştırılabilir.
Farklı kargo firma api'larına proje içerisinde istek atılması ve kullanılması gerekmesi.	Her kargo firmasının fiyat hesaplama ve takip no politikası farklıdır. Eğer bunları business katmanında doğrudan kullanırsak bu apilere bağımlı hale gelmiş olur ve sıkı bağımlı high coupling elde ederiz. İleride eklenebilecek yeni kargo firmaları için kodumuz firmalara bağımlı olmamalı. DIP ve OCP ihlali.	Adapter Pattern	Dependency Inversion ve low coupling gereği bütün kargo api'leri bizim iş katmanının koyduğu ortak bir interface'i gerçeklemelidir. Kargo firma api'lerinin metot imzalarını bizim interface'imize uyarlamak için aracı adaptor sınıfları yazarız.
Ödeme servisinin ve yöntemlerinin yapılması.	Ödeme yöntemi alınırken runtime zamanda üretilecek sınıfın değişmesi. Ödeme yöntemlerinin genişletilebilir olması gerekmesi.	Strategy Pattern	Polimorfizm'i kullanarak runtime zamanda ödeme yönteminin belirlenmesi. Modülün yöntemlerden bağımsız olması sağlanarak O/CP ilkesinin gerçekleşmesi sa

2. Zamanım kalsaydı uygulayacağım projeye uyacağını düşündüğüm desenler:

Süreç	Karşılaşılan Problem	Seçilen Tasarım Deseni	Neden bu desen?
Kargo Hizmetinin Seçilmesi ve ek fiyat hesabı.	Her hizmetin ayrı olarak seçilebilmesi ve her birinin ayrı kombinasyonlarıyla farklı sonuçlara yol açmaları.	Decorator Pattern	Wrapper mantığı ile içerisinde başka IKargo referansı tutabilen decorator sınıfları yazılır. Bu sayede her türlü kombinasyon otomatik olarak decorator sınıflarında hesaplanıp kullanılabilir.
Giriş yapan kişinin rolüne göre yapabileceği işlem/metotların kısıtlanması.	Şuanki projemde yetkisi olmayan viewlar direkt üretilmiyor fakat web uygulaması yapsaydım kötü niyetli insanlar zorla api isteği gönderip yetkisi olmadığı metotlara ulaşabilirlerdi.	Proxy Pattern	Araya koruyu bir proxy sınıfı girer. Bu sınıf kullanıcının rolüne göre direkt yetkisi olmayan metotları yasaklar, hata fırlatır.
Karmaşık ürünün inşası ve ilk kez üretilmesi.	Karmaşık ürünler üretilirken tek tek hafızada oluşturup alt birimleri içine eklemek zorunda kalıyoruz.	Builder Pattern	Her karmaşık ürünün içine ne eklenebileceği az çok bellidir çok daha anlaşılır bir syntax ile aşama aşama karmaşık ürün inşa edilir. (montaj yapılır)

